

Directed Graph Feedback Loop Stability & Memory Buffers

Real-Time DSP Safety Guarantees in Dynamic Modular Graphs

Time Dilation DAW (v4.0) Engineering Group

August 3, 2026

Abstract

This document formulates the directed graph cycle detection algorithms, topological sorting methods, and 1-block delay buffer history allocation mechanism ensuring 100% real-time audio thread stability in **Time Dilation DAW**.

1 Modular Audio Directed Graph Representation

The modular audio processing network is modeled as a Directed Graph $G = (V, E)$, where V represents audio and control processing nodes and $E \subseteq V \times V$ represents patch cable connections:

$$E = \{(u, v, i_{\text{out}}, j_{\text{in}}) \mid \text{Output } i_{\text{out}} \text{ of Node } u \text{ connects to Input } j_{\text{in}} \text{ of Node } v\} \quad (1)$$

2 Topological Sort & DFS Feedback Loop Detection

A feedback loop occurs when there exists a directed cycle $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$ in G . Depth-First Search (DFS) with node visitation states $\text{State}(v) \in \{\text{UNVISITED}, \text{VISITING}, \text{VISITED}\}$ identifies cyclic dependencies in linear $O(|V| + |E|)$ time complexity:

$$\text{IsFeedback}(u \rightarrow v) = \text{true} \iff \text{State}(v) == \text{VISITING} \quad (2)$$

3 1-Block History Delay Buffer Allocation

For any connection $e = (u, v) \in E$ identified as part of a feedback loop cycle:

$$x_{\text{dest}}[n] = y_{\text{src}}, \text{prevBlock}[n], \quad \forall n \in [0, K_{\text{block}} - 1] \quad (3)$$

where $y_{\text{src}}, \text{prevBlock}$ is the audio buffer rendered during block $N - 1$. For non-cyclic (causal forward) connections:

$$x_{\text{dest}}[n] = y_{\text{src}}, \text{currentBlock}[n], \quad \forall n \in [0, K_{\text{block}} - 1] \quad (4)$$

4 Mathematical Proof of Thread Safety & Zero Starvation

Let K_{block} be the audio buffer block size (e.g., 512 samples). Because all cyclic dependencies read from $y_{\text{src}}, \text{prevBlock}$, the input buffer contents x_{dest} for cyclic edges are completely determined prior to processing block N . Therefore, the dependency graph $G_{\text{acyclic}} = (V, E \setminus E_{\text{feedback}})$ forms a Directed Acyclic Graph (DAG), guaranteeing:

1. **Deterministic Topological Evaluation:** Node execution order is strictly well-ordered.
2. **Zero Audio Thread Deadlocks:** No processing node waits for uncalculated live samples within the same buffer quantum.
3. **Absolute Memory Boundedness:** Memory overhead per feedback edge is strictly limited to $1 \times K_{\text{block}}$ float array allocation.